CSCI6032 lecture 18 Random Forests

Decision Tree Review

Last time we covered decision trees.

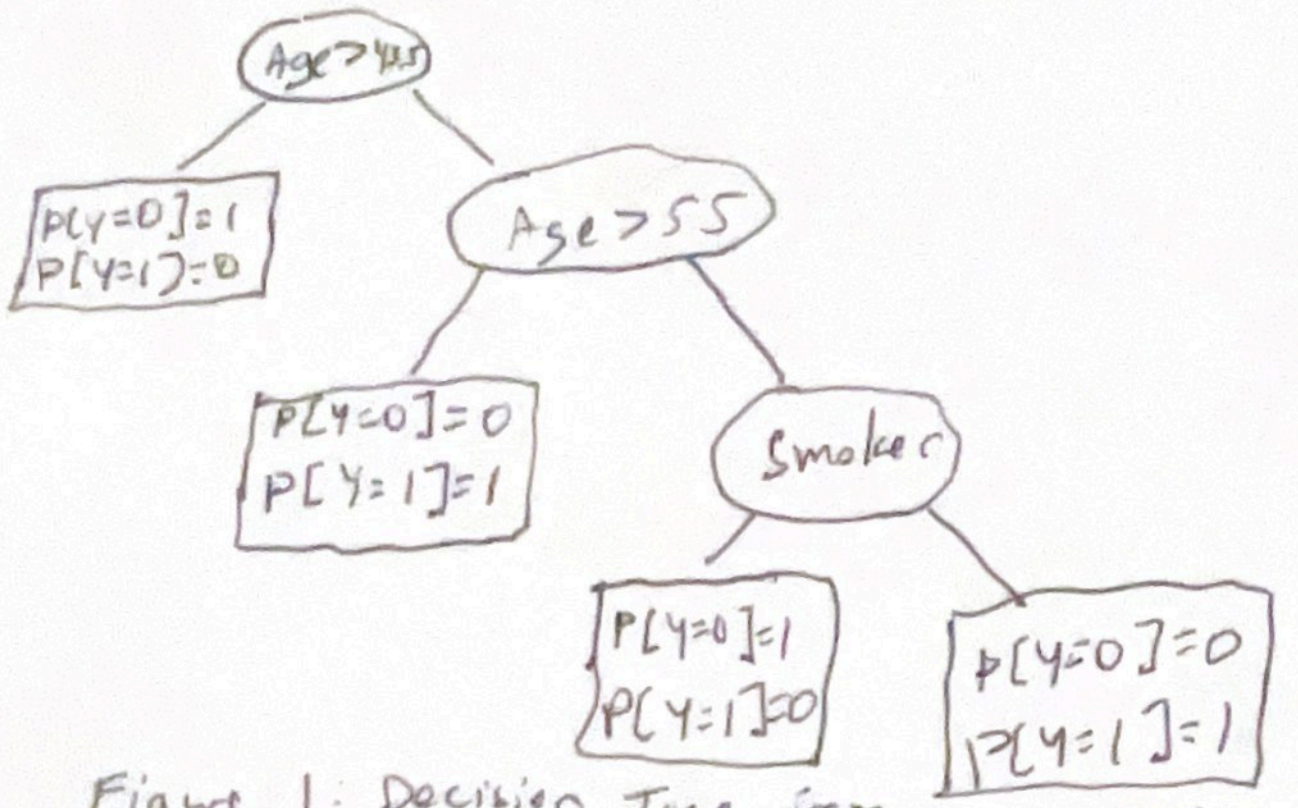


Figure 1: Decision Tree from prior lecture

With larger data sets, we often terminate when

- sample size gets too small
- Information Gain falls below a threshold min_gain but usually not zero.

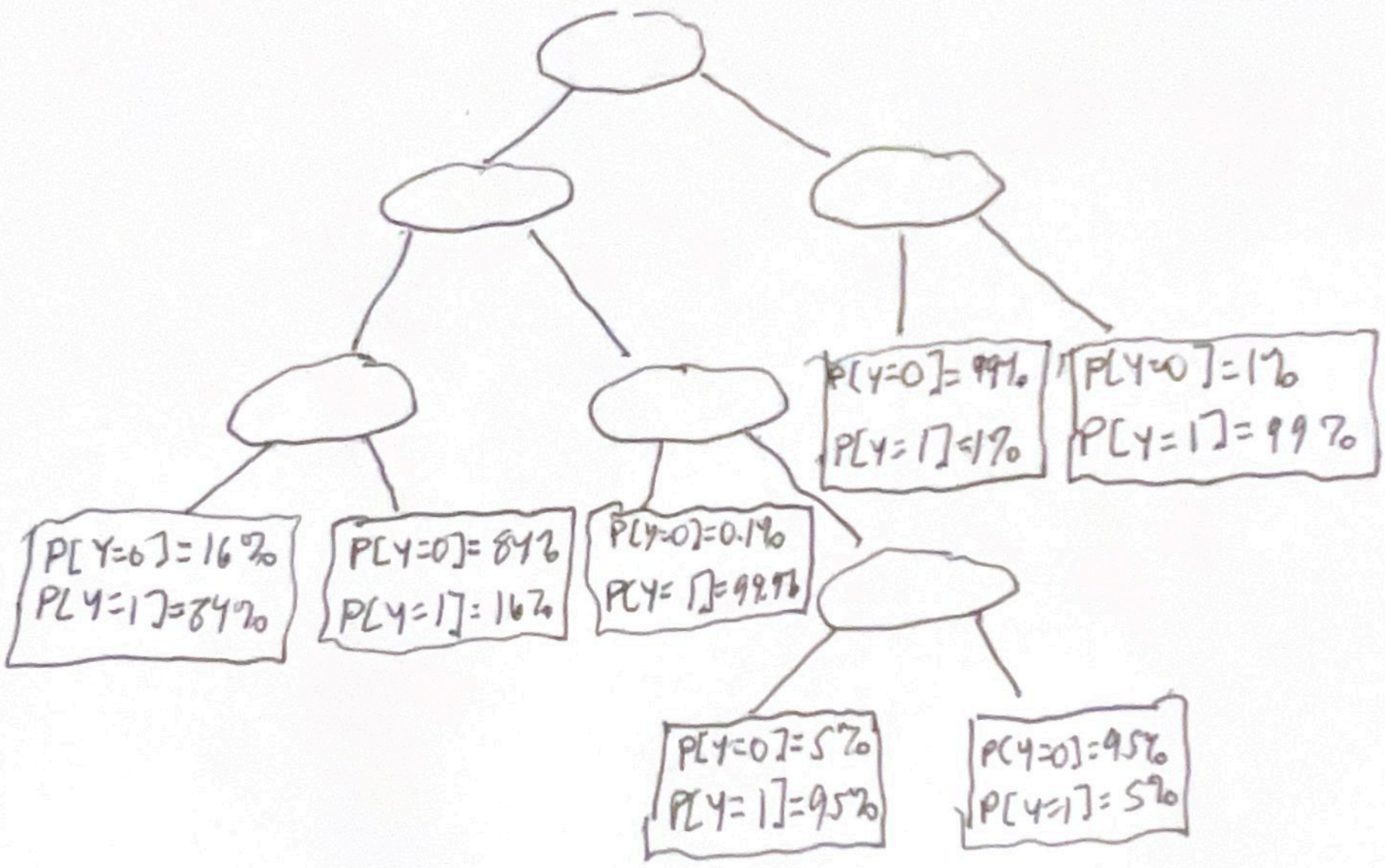


Figure 2: Example decision tree with non-zero posterior probabilities in leaves.

Decision Tree Review

Central idea is to always (greedily) pick the decision with maximum Information Gain (IG)

$$IG(s|X) = H(Y) - H(Y|s)$$

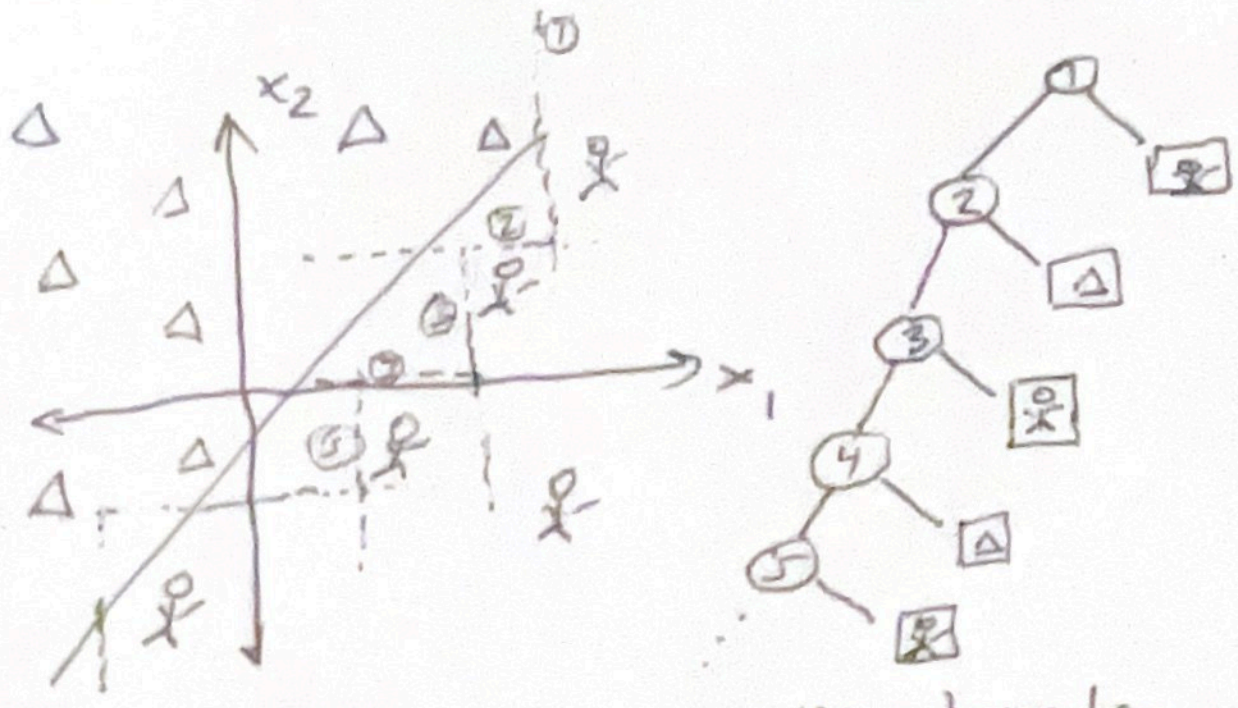
Information gain for split s on data set X.

Information gain measures the reduction in entropy given we consider the subsets partitioned by split s.

Limitations of Decision Trees

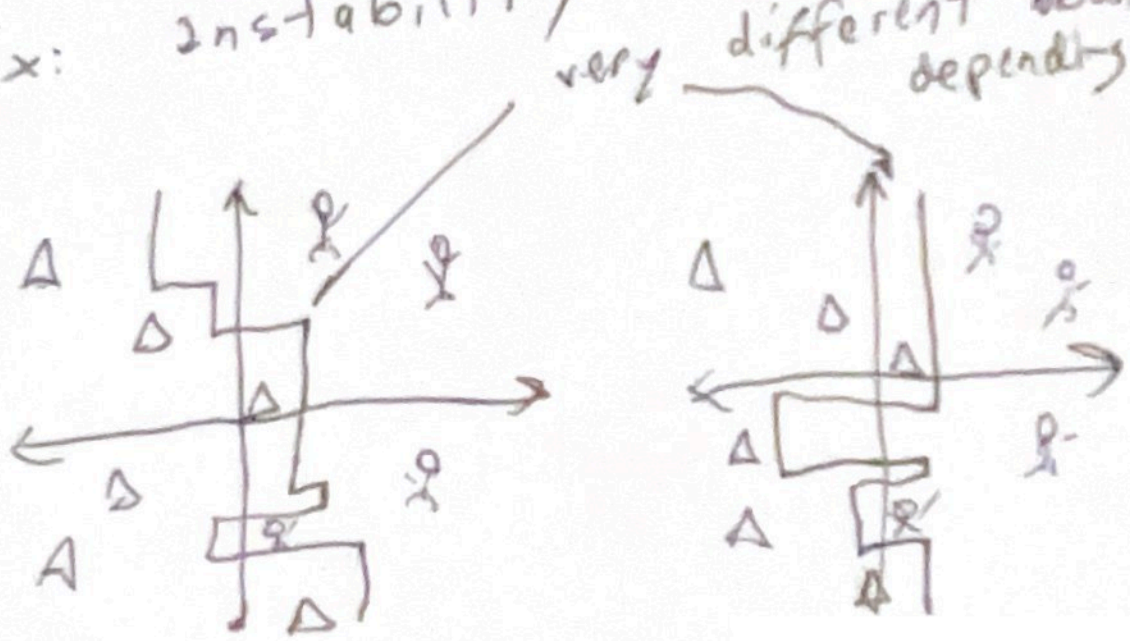
- Instability / high variance
  slight change in data sampled can result in a different order of splits resulting in ~~many~~ very different leaf posterior probabilities.
- Splits are aligned with axes.
  E.g. Age > 55 ← ~~divides~~ split data along the Age axis.
  By aligning with an axis we cannot capture decision boundaries not aligned with an axis without many decisions
- overfitting. Deep trees have small sample sizes near the leaves resulting in increasing variance or learning of the idiosyncracies of the data.
- ~~Leaf~~ leaf probabilities can be overconfident: another artifact of small sample sizes near the leaves.
- harder to interpret with depth.
  still better than neural networks which are famously opaque.

Ex: Problem w/ splits along an axis



To follow the off-axis decision boundary we end up creating decisions that follow a "staircase." This example has a linear boundary that fully separates the sets (i.e., the sets are "linearly separable"). We can describe this relationship with two parameters: slope and offset. To capture this with a decision tree requires many decisions.

Ex: Instability



(see demo in CSCI6032_26F_17_Decision_Trees.ipynb)

Bootstrapping

We use bootstrapping when n (sample size) is limited and we don't know the underlying distribution.

Bootstrap = resampling with replacement original sample. $n=5$

1 4 9 11 17

To create a bootstrap sample we pick 5 with replacement, e.g.,

9 4 11 11 4

Another bootstrap sample

17 1 9 4 9

The same value can occur more than once.

If we care about order then we can create n^n bootstrap samples from the original samples.

If we don't care about order then

9 4 11 11 4

is the same as

4 11 4 9 11

$\binom{2n-1}{n}$ ~~possible~~ distinct bootstrap resamples.

bootstrapping (cont.)

$\binom{2n-1}{n}$

1 3 5 ← A sample set is really a bag.

* * | | * ↘
1 1 5

* = a value drawn from the original bag.

Aside: A bag is like a set but it can contain more than one of the same value.

||| *** ↘
5 5 5

* | * | * ↘
1 3 5

Since there are n samples in the original sample set, there must be n *'s.
Each of the samples can take on 1 of n possible values so we have n-1 dividers (bars).
we have n+n-1 stars and bars of which n-1 of these n+n-1 spots can be a bar.
so $\binom{2n-1}{n}$.

Random Forest

Training:

1. Create a bootstrap sample X' from X.
2. Create a decision tree from X'
   but at each node in the decision tree pick the feature and split with max information gain from a random subset of features.
3. Repeat 1 & 2 until we have J trees.

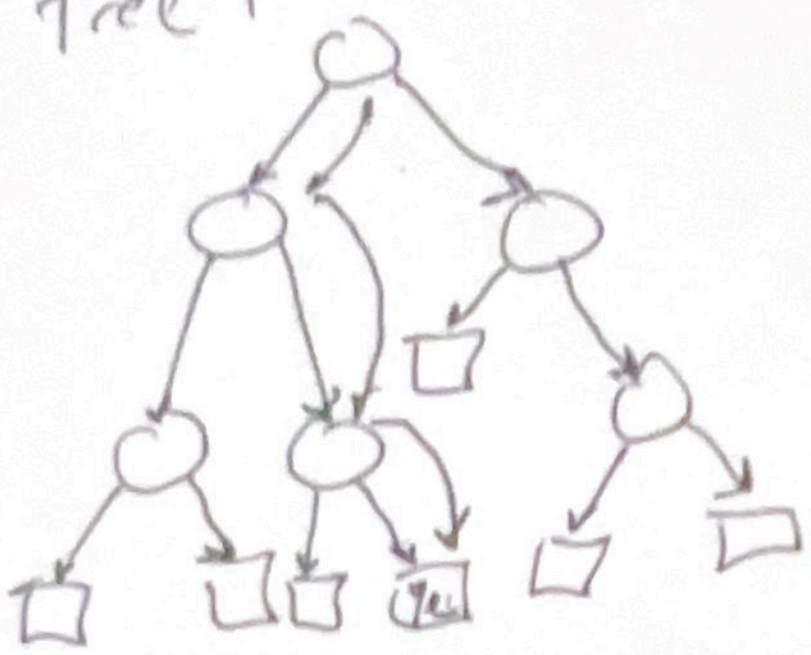
often use $\sqrt{n}$ randomly chosen features, where n = # features in $\vec{x}$.

Random Forest (cont.)
Prediction

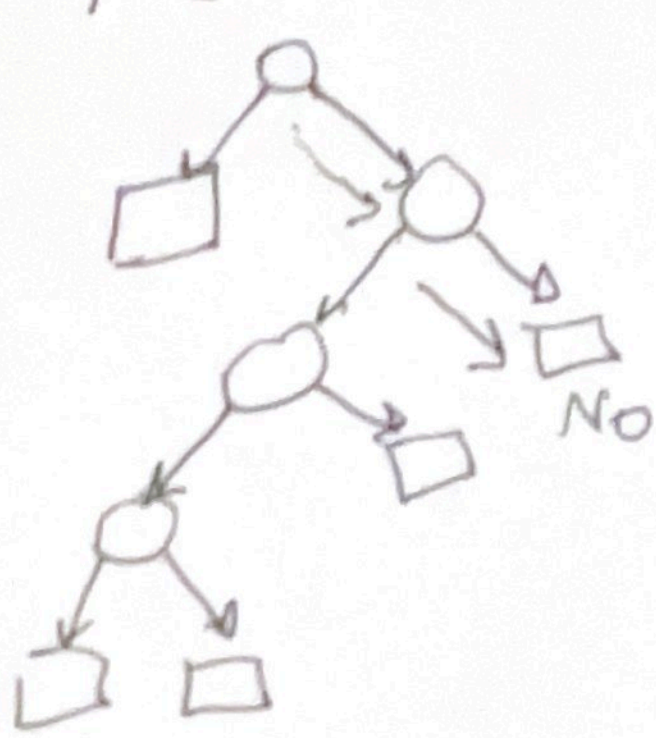
1. Apply all T decision trees on the query data point $\bar{x}$. (feature vector)
2. A Perform majority vote.

Ex:

Tree 1



Tree 2



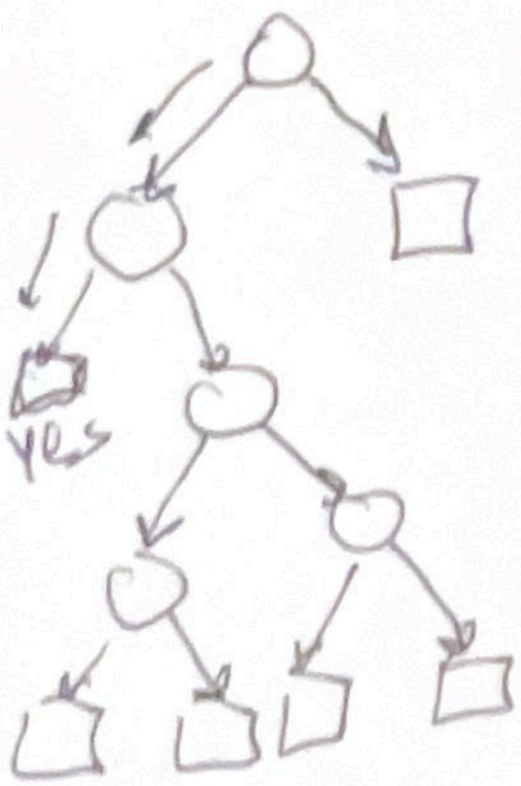
Total votes

yes 2

no 1

yes wins!

Tree 3



Advantages of Random Forest

- Much lower variance
- usually higher accuracy
- can use samples that were not selected in the resample for validation. The unselected samples are called "Out-of-bag" which is typically about ~~36.7~~ 36.8% per tree.
- more robust to outliers than a single decision tree.
- parallelizable. Can train each tree independently.